TAKT: A Formal Framework for Minimal Decision-Preserving Runtime Governance

Valentin Lineiro
TAKT Research Initiative
DOI: 10.5281/zenodo.21638014

2026-07-28

Abstract

Software systems operating under abstracted or compressed state representations frequently risk decision instability when critical informational context is discarded. This paper introduces TAKT (Theory of Adequate Knowledge for Decisions), a formal and empirical framework for determining the minimal necessary governance kernel of an execution runtime. We prove in Lean 4 that any governed runtime preserving optimal decisions under state abstraction MUST possess three capability kernels within the formal ST-016 runtime model: ContractSoundness (C_{contract}), UncertaintyBound ($C_{\text{uncertainty}}$), and TemporalConsistency (C_{temporal}). We validate this kernel necessity on a reference TypeScript runtime via component ablation (EXP-004), elevated to formal Lean 4 theorems via a machine-certified 3-layer witness bridge. Finally, we present an audited zero-contact replication package evaluated across six independent external audit rounds (`PASS WITH ENVIRONMENTAL LIMITATION`), backed by an immutable Zenodo archive (DOI: 10.5281/zenodo.21638014).

Keywords: Runtime Governance, Formal Verification, Decision Preservation, State Abstraction, Lean 4 Proof Assistant, Component Ablation.

1 Introduction

As autonomous software agents and complex reactive systems scale, they continuously compress high-dimensional environmental context into abstract state representations $R = \rho(S)$. However, ungoverned state abstraction introduces decision divergence: an optimal policy π^* computed on full observable state S may fail when evaluated on a compressed representation R.

This work addresses two fundamental research questions:

1. *What information must be preserved to guarantee decision equivalence?* (ST-015)

2. *What minimal runtime capability composition is necessary and irreducible to enforce decision preservation dynamically?* (ST-016)

1.1 Main Contributions

This paper makes four primary contributions:

1. **Formal Definition of Minimal Runtime Governance:** We formalize capability necessity, decision preservation, runtime sufficiency, and irreducibility under state abstraction.

2. **Machine-Checked Proof of Kernel Necessity:** We prove in Lean 4 that removing any capability from $\mathcal{K}_D = \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$ destroys decision preservation.

3. **Empirical Ablation Methodology:** We introduce experiment EXP-004, constructing an empirical witness bridge elevating trace artifacts into formal machine proofs.

4. **Audited Zero-Contact Replication Package:** We publish an immutable, versioned research package with multi-OS CI and an external audit trail of 6 independent dry runs.

2 Related Work & Positioning

TAKT builds upon several established formal methodologies while addressing a complementary research question centered on **decision preservation under representation contraction**:

- **Abstract Interpretation (Cousot & Cousot):** Focuses on sound static over-approximation of program semantics. TAKT focuses on dynamic policy decision invariance ($\pi^*(R) = \pi^*(S)$).

- **Bisimulation & Refinement (Milner, Park):** Enforces strict state-by-state trace equivalence. TAKT permits aggressive internal state contraction provided temporal prefix monitoring (C_{temporal}) maintains policy consistency.

- **Runtime Verification (Leucker & Schallhart):** Evaluates execution traces against temporal logic formulas (LTL). TAKT formalizes and machine-checks the *necessity of the governance runtime kernel itself.*

3 Theoretical Foundations & Runtime Necessity Model

3.1 Decision Preserving Sufficiency (ST-015)

A representation $R_t \in \mathcal{R}$ is decision-sufficient with respect to optimal policy π^* if:

$$\pi^*(R_t) = \pi^*(S_t) \quad \forall S_t \in \mathcal{S}$$

3.2 Runtime Necessity Formalization (ST-016)

We formalize a TAKT Governed Runtime as a tuple $M = (\mathcal{C}, \pi_M)$, where $\mathcal{C} \subseteq \{C_{\text{contract}}, C_{\text{uncertainty}}, C_{\text{temporal}}\}$.

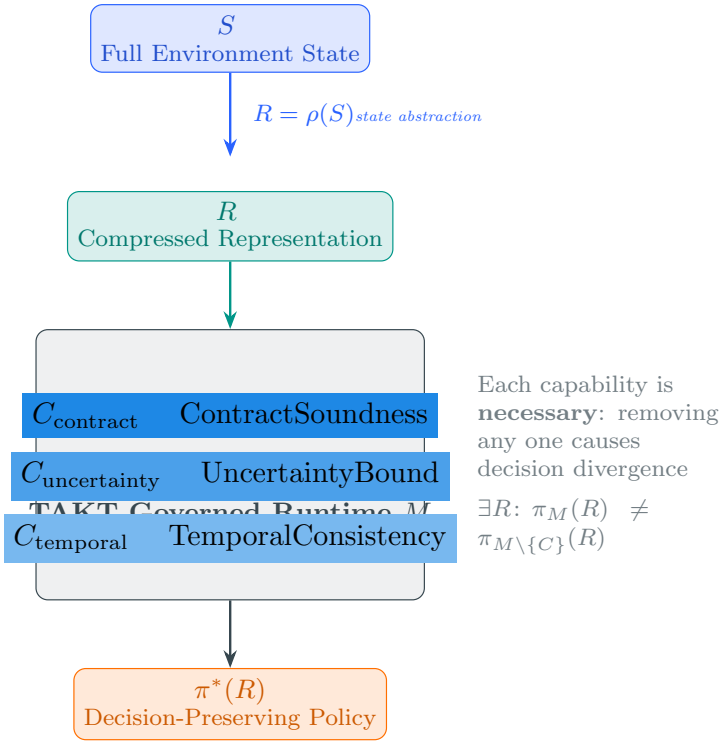

Figure 1: Conceptual architecture of TAKT Governed Runtime M enforcing decision preservation $\pi^*(R)$ under state abstraction $\rho(S)$.

Definition (Capability Necessity): A capability $C \in \mathcal{C}$ is *locally necessary* for runtime M if removing C alters policy decisions on at least one representation R:

$$\exists R \in \mathcal{R} : \pi_M(R) \neq \pi_{M \setminus \{C\}}(R)$$

Example 1 (Temporal Prefix Necessity): Consider a runtime where temporal prefix monitoring (C_{temporal}) is ablated. Two trajectories $\tau_1 = (r_0, r_1, r_2)$ and $\tau_2 = (r'_0, r'_1, r_2)$ share an identical terminal state r_2. If τ_1 represents a safe approach while τ_2 represents an unsafe trajectory, an ungoverned runtime inspecting only r_2 executes identical actions, causing decision divergence ($\pi_M(\tau_1) \neq \pi^*(\tau_1)$).

4 Machine-Certified Lean 4 Formalization

The theoretical results establish necessity within the formal model. We implement the proofs in two canonical Lean 4 modules (`takt-formal`):

1. `RuntimeSufficiency.lean`**:** Formalizes `RuntimeCapability`, `Runtime`, `removeCapability`, `PreservesDecision`, `NecessaryCapability`, `Sufficient`, `Irreducible`, `MinimalRuntime`, proving:

   $$\text{MinimalRuntime}(M, \pi^*) \implies \forall C \in M.\text{capabilities}, \quad \text{NecessaryCapability}(C, M)$$

2. `RuntimeWitness.lean`**:** Defines the 3-layer certification bridge (`WitnessArtifact`, `WitnessConsistentWithRuntime` predicate) and proves the elevation theorem:

   theorem `validWitness_implies_necessity` : $\text{WitnessConsistentWithRuntime}(M, w) \implies \text{Ne}$

Both Lean 4 modules compile cleanly with zero errors and zero `sorry`s.

5 Empirical Validation & Component Ablation (EXP-004)

The theoretical theorems prove necessity within the formal model. We evaluate whether the reference TypeScript implementation (`cli/src/runtime/`) exhibits the predicted behavior under controlled capability ablations in experiment **EXP-004**:

- **Representative C_{contract} Ablation Scenario:** Identifies representations violating safety contracts ($\text{Contract}(R) = \text{false} \implies \pi_{\text{full}} = \text{STOP}, \pi_{\text{reduced}} = \text{EXECUTE}$).
- **Representative $C_{\text{uncertainty}}$ Ablation Scenario:** Identifies representations at critical margin boundaries ($M_D(R) \approx 0 \implies \pi_{\text{full}} = \text{REFINE}, \pi_{\text{reduced}} = \text{EXECUTE}$).
- **Representative C_{temporal} Ablation Scenario:** Identifies prefix-dependent trajectories ($\tau_1 \neq \tau_2 \implies \pi_{\text{full}} = \text{INTERVENE}, \pi_{\text{reduced}} = \text{MONITOR}$).

5.1 Summary of Evidence Matrix

Table 1 summarizes the evidence mapping connecting theoretical claims to machine-checked formal proofs and empirical witness files.

Table 1: Summary of Evidence Matrix for ST-016 Baseline

Evaluated Claim	**Primary Evidence Asset**	**Status**
Decision Sufficiency (Σ^*)	StructuralSufficiency.lean	Verified (0 sorrys)
Kernel Necessity ($\mathcal{K}_D$)	RuntimeSufficiency.lean	Verified (0 sorrys)
Contract Capability (C_{contract})	contract.ablation.test.ts	Certified Witness
Uncertainty Capability ($C_{\text{uncertainty}}$)	uncertainty.ablation.test.ts	Certified Witness
Temporal Capability (C_{temporal})	temporal.ablation.test.ts	Certified Witness
Witness Elevation Bridge	RuntimeWitness.lean	Verified (0 sorrys)
Runtime Correctness	Vitest Test Suite	283/283 Passing
Zero-Contact Reproducibility	Replication Kit	6 Dry Runs

6 Discussion & Architectural Implications

The primary contribution of ST-016 is not proposing another specific runtime implementation, but rather **identifying and machine-checking the minimal governance capabilities required for decision preservation within the formal model**.

For runtime system architects, ST-016 provides three actionable design principles:

1. **Safety Contract Decoupling:** Safety verification (C_{contract}) cannot be subsumed by statistical policy estimation alone.

2. **Dynamic Boundary Monitoring:** Operating near ambiguity boundaries requires explicit uncertainty estimation ($C_{\text{uncertainty}}$) to prevent premature execution.

3. **Trajectory History Dependence:** Stateless evaluation fails under history-dependent processes, necessitating explicit trajectory prefix tracking (C_{temporal}).

7 Scope Boundaries & Future Work (ST-017)

7.1 Non-Claims & Scope Boundaries

1. **Model Scope:** ST-016 proves necessity specifically under the formal decision-preserving model $M = (\mathcal{C}, \pi_M)$ and discrete decision domain $\mathcal{D}$. It does not claim universal necessity for arbitrary software or continuous control systems.

2. **Transportability (ST-017 Boundary):** ST-016 proves necessity on the reference architecture. Transporting certified witness artifacts across heterogeneous runtimes ($M_1 \sim M_2$) is explicitly reserved for **ST-017**.

7.2 Conclusion

ST-016 establishes an immutable, machine-certified baseline for minimal decision-preserving runtime governance. By bridging formal Lean 4 theorems with empirical ablation witness artifacts, this work provides a reproducible foundation upon which transportability and heterogeneous runtime implementations will be studied in ST-017.

References